

Flexbox Layout

One-Dimensional Layouts Made Easy

■ FLEXBOX

■ LEARNING OUTCOME

By the end of this module you will centre anything on a page, build navigation bars, card grids, and sidebar layouts — all using Flexbox — and understand the difference between main axis and cross axis control.

■ Skills You Will Gain

- Activate Flexbox and understand container vs item properties
- Control main axis alignment with justify-content
- Control cross axis alignment with align-items and align-self
- Use flex-wrap, gap, and order for responsive layouts
- Apply flex-grow, flex-shrink, and flex-basis to items
- Build navbar, card grid, and holy grail layout with Flexbox

■ Flexbox replaced floats, tables, and inline-block hacks overnight. It's used in 90% of modern layouts. Master it in one session and layout problems disappear.

SECTION 1

Flexbox Core Concepts

Container vs Items

Flexbox has TWO levels: FLEX CONTAINER: the parent element — receives display:flex. Controls overall layout. FLEX ITEMS: the direct children of the container. Can override individually. Main Axis: the PRIMARY direction of flex items (row = horizontal, column = vertical) Cross Axis: PERPENDICULAR to main axis (row = vertical, column = horizontal) justify-content: controls alignment along the MAIN axis align-items: controls alignment along the CROSS axis

```
/* ===== FLEX CONTAINER PROPERTIES ===== */ .container { display: flex; /* or inline-flex */ /*
Direction of main axis */ flex-direction: row; /* DEFAULT: left to right */ flex-direction:
row-reverse; /* right to left */ flex-direction: column; /* top to bottom */ flex-direction:
column-reverse; /* Main axis alignment (horizontal when flex-direction: row) */ justify-content:
flex-start; /* DEFAULT: items at start */ justify-content: flex-end; /* items at end */
justify-content: center; /* items centred */ justify-content: space-between; /* equal gaps BETWEEN
items */ justify-content: space-around; /* equal gaps AROUND items */ justify-content:
space-evenly; /* equal gaps including edges */ /* Cross axis alignment (vertical when
flex-direction: row) */ align-items: stretch; /* DEFAULT: items fill container height */
align-items: flex-start; /* items at top */ align-items: flex-end; /* items at bottom */
align-items: center; /* items vertically centred */ align-items: baseline; /* align by text
baseline */ /* Allow items to wrap to new rows */ flex-wrap: nowrap; /* DEFAULT: all items on one
line */ flex-wrap: wrap; /* wrap to new rows */ flex-wrap: wrap-reverse; /* wrap in reverse order
*/ /* Space between rows (only when wrapping) */ align-content: flex-start | center | space-between
| space-evenly; /* Modern: gap between items (no margin hacks!) */ gap: 16px; /* same for rows and
columns */ gap: 16px 24px; /* row-gap column-gap */ row-gap: 16px; column-gap: 24px; }
```

SECTION 2

Flex Item Properties

```
/* ===== FLEX ITEM PROPERTIES ===== */ .item { /* flex-grow: how much item GROWS relative to
siblings when there's extra space */ flex-grow: 0; /* DEFAULT: don't grow */ flex-grow: 1; /* grow
equally with other flex-grow:1 items */ flex-grow: 2; /* grow twice as fast as flex-grow:1 items */
/* flex-shrink: how much item SHRINKS when not enough space */ flex-shrink: 1; /* DEFAULT: shrink
equally */ flex-shrink: 0; /* never shrink – item keeps its size */ /* flex-basis: initial size
BEFORE growing/shrinking */ flex-basis: auto; /* DEFAULT: use width/height */ flex-basis: 200px; /*
start at 200px */ flex-basis: 30%; /* start at 30% of container */ flex-basis: 0; /* start at 0,
distribute space from flex-grow */ /* SHORTHAND: flex: grow shrink basis */ flex: 0 1 auto; /*
DEFAULT */ flex: 1; /* flex: 1 1 0% – grow equally, shrink equally */ flex: none; /* flex: 0 0 auto
– don't grow or shrink */ flex: 0 0 200px; /* fixed 200px – no grow, no shrink */ flex: 1 1 300px;
/* min 300px, grow if space available */ /* Override cross axis for this item only */ align-self:
auto | flex-start | flex-end | center | stretch | baseline; /* Visual reorder (does NOT change
tab/reading order – accessibility risk!) */ order: 0; /* DEFAULT: source order */ order: -1; /*
move to front */ order: 1; /* move toward end */ /* Prevent item from shrinking below its
min-content size */ min-width: 0; /* often needed inside flex containers */ }
```

SECTION 3

Practical Flexbox Patterns

```
/* ===== 1. PERFECT CENTRING (both axes) ===== */ .center-both { display: flex; justify-content:
center; align-items: center; min-height: 100vh; } /* ===== 2. NAVIGATION BAR ===== */ .navbar {
display: flex; justify-content: space-between; /* logo left, links right */ align-items: center;
padding: 0 24px; height: 64px; background: white; box-shadow: 0 1px 3px rgba(0,0,0,.1); }
```

```

.navbar__links { display: flex; gap: 32px; list-style: none; } /* ===== 3. CARD GRID ===== */
.card-grid { display: flex; flex-wrap: wrap; gap: 24px; } .card { flex: 1 1 280px; /* grow, shrink,
min-width 280px */ /* Automatically creates columns that wrap to new rows */ } /* ===== 4. PUSH
LAST ITEM TO END ===== */ .header { display: flex; align-items: center; gap: 16px; } .header__cta {
margin-left: auto; /* pushes this item to the far right */ } /* ===== 5. EQUAL HEIGHT COLUMNS =====
*/ .columns { display: flex; /* align-items: stretch by DEFAULT */ } .column { flex: 1; } /* each
column takes equal width */ /* ===== 6. SIDEBAR LAYOUT ===== */ .page-layout { display: flex;
min-height: 100vh; } .sidebar { flex: 0 0 280px; } /* fixed 280px width */ .content { flex: 1; } /*
takes remaining space */

```

SECTION 4

Common Flexbox Mistakes

Mistake	Problem	Fix
Using margin for gaps	Margin on first/last item adds unwanted edge space	Use gap property instead
Forgetting flex-wrap	Items overflow on small screens	Add flex-wrap: wrap
Using order for semantics	Changes visual order but not tab/reading order	Reorder HTML source instead
min-width:0 missing	Long text overflows flex items	Add min-width:0 or overflow:hidden to items
Nested flex confusion	Flex properties only affect direct children	Add display:flex to every level that needs it

■ PRACTICE Q & A

Q1. What is the difference between justify-content and align-items?

A: justify-content: aligns items along the MAIN axis (horizontal in flex-direction:row). align-items: aligns items along the CROSS axis (vertical in flex-direction:row). Think: justify = main axis, align = cross axis.

Q2. What does flex: 1 mean?

A: Shorthand for flex: 1 1 0%. The item can grow (1), can shrink (1), and starts at 0 base size. Multiple items with flex:1 divide available space equally.

Q3. How do you push one flex item to the far right while keeping others on the left?

A: Add margin-left: auto to the item you want pushed right. The auto margin absorbs all available space, pushing the item to the opposite end. No extra wrappers needed.

Q4. What is the difference between align-items and align-self?

A: align-items is set on the CONTAINER and applies to ALL items. align-self is set on an individual ITEM and overrides align-items for just that one item.

Q5. When should you use flex-wrap: wrap?

A: When you want items to flow onto new rows when there's not enough space — creating a responsive grid without media queries. Combined with flex: 1 1 minWidth, items automatically create as many columns as fit.

■ Flexbox Cheat Sheet	
display: flex	Activate Flexbox on container
flex-direction: row/column	Main axis direction
justify-content: center	Align on MAIN axis
justify-content: space-between	Space between items

align-items: center	Align on CROSS axis
flex-wrap: wrap	Items wrap to new rows
gap: 16px	Space between items – no hacks
flex: 1	Item grows to fill space
flex: 0 0 200px	Fixed 200px – no grow/shrink
align-self: flex-end	Override alignment for one item
margin-left: auto	Push item to far end
order: -1	Move item visually first